

A Coherent Foundation for Memory-Bounded CoinVault Knowledge Pipelines

From MySQL snapshot to immutable index generation, verified serving, and
CoinVault tools

Architecture and implementation analysis of CoinVault and RagKit

2026-08-12

Contents

Executive summary	4
1. Scope, method, and evidence	6
1.1 Scope	6
1.2 Material analyzed	7
1.3 Terminology	7
2. Current end-to-end system	8
2.1 Source boundary	8
2.2 Historical eager build	8
2.3 Current staged build repair	9
2.4 Bundle format and verification	9
2.5 Serving and tool path	10
2.6 Operational and command surface	10
3. Evidence from the memory work	11
3.1 Failure and repair timeline	11
3.2 What the measurements prove	11
3.3 What remains unproven	12
4. The actual root cause: implicit liveness and duplicated lifecycle	12
4.1 Memory is a property of schedules, not just functions	12
4.2 “Streaming” is underspecified	13
4.3 The current abstractions stop at the wrong boundaries	13
4.4 Product lifecycle is split into parallel state machines	13
5. Mathematical and computer-science foundations	14
5.1 External-memory model	14
5.2 Iterator/Volcano execution	14
5.3 Synchronous dataflow and static schedules	15
5.4 Queueing theory and backpressure	15
5.5 Monoids and streaming identities	15
5.6 Typestate and legal phase transitions	16
5.7 Algebraic effects and ports/adapters	16
6. A precise boundedness model	16
6.1 Boundedness lattice	16
6.2 Resource vectors, not one memory number	17
6.3 Static live-set estimate	17

6.4 Runtime leases	18
6.5 Go soft limit and cgroup hard limit	18
7. Target architecture	19
7.1 Six layers	19
7.2 One semantic generation, several physical views	19
7.3 Logical versus physical nodes	20
7.4 Compiler passes	20
8. The end-to-end CoinVault physical plan	21
8.1 Snapshot and source receipt	21
8.2 Streaming products and facets with a merge join	22
8.3 Normalization and exclusion as one pass	22
8.4 Document-local chunking	23
8.5 Chunk-local representations	23
8.6 Embedding as a bounded asynchronous island	24
8.7 Seal before backend construction	24
8.8 Build backends sequentially from the same relations	25
8.9 Local scratch, immutable publication, and EFS	25
8.10 Verification profiles	25
8.11 Activation and rollback	26
9. Serving as a resource-bounded pipeline	26
9.1 Current query plan	26
9.2 Per-query bound	27
9.3 Exact vector search	27
9.4 Authorization as a relational operator	27
9.5 Reranking and provider boundaries	28
9.6 Hydration and evidence	28
10. The compositional kernel	28
10.1 Minimal Go contracts	28
10.2 Ownership rule	29
10.3 Operator descriptor	29
10.4 Relation descriptor	29
10.5 Materialization is explicit	29
10.6 No implicit goroutine graph	30
10.7 Unified run events	30
10.8 Checkpoint and resume	31
11. A practical DSL	31
11.1 Go builder is authoritative	31
11.2 Restricted YAML authoring layer	31
11.3 Physical plan report	33
12. Plugin and package architecture	33
12.1 Repository ownership	33
12.2 Plugin classes	34
12.3 Extension tiers	34
12.4 Conformance suites	34
13. Identity model	34
13.1 Separate identities	34
13.2 Canonical record encoding	35
13.3 Plan lockfile	35
14. Verification as compiled invariants	36
14.1 Invariant classes	36

14.2 Differential validation	36
14.3 Fail closed on incomplete evidence	36
15. Observability without a parallel architecture	36
15.1 One event stream	36
15.2 Measurement hierarchy	36
15.3 Cardinality and stage correlation	37
15.4 Privacy	37
16. Engineering workflow	37
16.1 Plan-first change process	37
16.2 CI tiers	37
Fast PR checks	37
Bounded integration checks	38
Scheduled provider-backed checks	38
Release checks	38
16.3 Memory regression tests	38
16.4 Crash testing	38
16.5 Documentation workflow	39
17. Integration with self-optimization	39
18. Migration plan	40
18.1 Phase 0 — Freeze contracts and delete ambiguity	40
18.2 Phase 1 — Minimal kernel around existing RagKit	40
18.3 Phase 2 — Stream MySQL into the document relation	40
18.4 Phase 3 — Remove eager chunks and representations	40
18.5 Phase 4 — Consolidate generation storage	41
18.6 Phase 5 — Correct storage placement and publication	41
18.7 Phase 6 — Service resource governor	41
18.8 Phase 7 — Unify refresh and promotion	41
18.9 Phase 8 — Remove legacy paths	41
19. Concrete first vertical slice	41
20. Acceptance criteria for the full foundation	42
Correctness	42
Memory and resources	42
Extensibility	42
Operations	42
Serving	43
21. Risks and anti-patterns	43
21.1 Building a universal workflow engine	43
21.2 A goroutine per node	43
21.3 One-row-at-a-time dogma	43
21.4 GOMEMLIMIT as the design	43
21.5 Measuring only terminal RSS	43
21.6 Writing staging SQLite on EFS	43
21.7 Keeping two semantic sources	43
21.8 Hiding sorts and maps inside transforms	43
21.9 Unlimited service concurrency	44
21.10 Hot reload before release custody	44
21.11 A Turing-complete YAML DSL	44
21.12 Treating telemetry as authorization	44
22. Final recommendation	44

Appendix A. Suggested Go types	45
Appendix B. Example compiled node	46
Appendix C. Current source map	46
CoinVault source and serving	46
RagKit mechanisms	47
Operational evidence	47
Appendix D. Design influences and references	47
Appendix E. Concrete current-to-target refactoring map	48
Appendix F. Retention and work ledger	49
Appendix G. Plan and code-review checklist	49
Appendix H. Additional references	50

Executive summary

CoinVault has already solved several real memory failures correctly. The original eager build retained the corpus, stripped corpus, chunks, representations, and every 1,536-dimensional vector while beginning additional lexical and vector backend construction. It completed 114,106 embeddings in both a 2 GiB and an 8 GiB Fargate task, then died at the backend handoff. The replacement RagKit BuildStream path stages bounded batches in SQLite and lets the content, Bleve, and vector backends consume ordered cursors. A production-shaped synthetic run peaked near 264 MiB RSS, and the real cache-backed build completed with a measured container maximum near 1.1 GiB. The verifier was then independently converted from eager decoded arrays and an all-hit Bleve result set to streaming JSON validation, compact identity maps, paged lexical inspection, and the already-streaming SQLite vector inspector. On the exact bundle, charged cgroup memory fell from roughly 2.0 GiB to roughly 90 MiB, while preserving the same schema and bundle identity.

Those are strong results. The remaining problem is architectural rather than algorithmic. Each pressure point was discovered and repaired through a different path:

- the product connectors load MySQL rows into product-specific slices and maps;
- the CoinVault build command creates full document, chunk, and representation collections before crossing the new staged boundary;
- RagKit has separate eager build, staged build, streaming verify, serving open, inspect, statistics, content-store, lexical, and vector implementations;
- CoinVault adds separate memory recorders, samplers, log and EMF sinks, cache rebuild/export, bundle export, state inspection, packaging, evaluation, sweep, judge, refresh, and deployment workflows;
- production refresh and promotion have another state vocabulary and another coordinator design;
- serving has a good bounded query path, but its resource bounds are implicit constants rather than a compiled and enforced request contract.

The disparate work has a common cause: **data ownership, order, persistence, resource use, and lifecycle effects are not represented in one executable plan**. The code often knows *what* each stage computes, but not in a machine-checkable way:

- whether a stage retains its input;
- what order it requires and produces;
- whether it can replay;
- where it may spill;
- how much memory or concurrency it reserves;

- whether it mutates external state;
- what digest proves semantic equivalence;
- what durable checkpoint makes resume legal;
- what release receipt authorizes serving.

The recommended foundation is a deliberately small **typed external-memory pipeline kernel**, not a general distributed workflow system. Its semantic center is an ordered, immutable set of typed relations—documents, exclusions, chunks, representations, vectors, and receipts—backed by a local scratch relation store during construction and by immutable bundle stores after publication. A logical plan says what transformations and invariants compose the CoinVault knowledge system. A compiler chooses a physical strategy for every edge: direct cursor, bounded batch, merge join, external sort, SQLite materialization, index-native scan, or bounded top-k. An executor owns memory leases, cancellation, checkpoints, events, atomic sealing, and artifact custody.

The end-to-end target is:

```
MySQL read-only repeatable-read snapshot + curated SQL docs
  |
  v
ordered source cursors
  |
  +--> streaming merge of products and product_details
  |
  v
normalized document relation + exclusion relation
  |
  v
per-document chunk cursor
  |
  v
per-chunk representation cursor
  |
  v
bounded cache lookup / provider batches
  |
  v
vector relation
  |
  v
sealed semantic generation plan
  |
  +--> content.sqlite
  +--> Bleve
  +--> vectors.sqlite
  +--> structural verification
  |
  v
immutable generation + verification/publication receipts
  |
  v
atomic channel activation
  |
  v
read-only serving handles
  |
  v
bounded lexical/vector candidates -> authorization -> fusion
  -> bounded reranking -> bounded hydration -> knowledge_search
  -> CoinVault chat runtime
```

The framework should make four different memory claims explicit:

1. **Strictly bounded state:** independent of corpus cardinality, such as a batch buffer, one document, one vector, or a top-k heap.
2. **Parameter-bounded state:** bounded by declared configuration such as batch size, dimensions, reranker pool, concurrent queries, or evidence budget.
3. **Cardinality-bounded compact state:** proportional to record identities rather than payloads, as in the current streaming verifier’s ID/digest maps.
4. **Unbounded/eager state:** proportional to corpus payload or vector coordinates and therefore illegal in a bounded production plan unless the compiler can prove it fits a declared finite dataset ceiling.

The core operational invariant should be:

$$M_{peak} \leq M_{base} + \sum_{r \in live} reservation(r) + M_{external} + \epsilon$$

where $reservation(r)$ covers owned Go buffers and bounded operator state, $M_{external}$ covers SQLite/Bleve/native/page-cache effects that the Go runtime does not fully control, and ϵ is measured safety margin. A stage cannot start until the shared resource governor grants its lease. GOMEMLIMIT remains a secondary guard, not the proof of boundedness.

The most important concrete changes are:

- replace LoadProductDocuments and loadProductFacets with ordered cursor sources and a streaming merge join, reducing facet state from all products to one product group;
- make CoinVault emit documents, chunks, representations, and vectors directly into a relation writer rather than first constructing complete slices;
- make the relation store the single canonical intermediate source for index construction, verification, inspection, statistics, packaging, and resume;
- place writable staging SQLite on local ephemeral storage, not EFS; publish only closed immutable files to EFS/S3, with a receipt written last;
- unify build, verify, open, inspect, export, refresh, and promote under one Plan -> Compile -> Execute -> Seal -> Verify -> Publish -> Activate protocol;
- define one event schema and sink adapter layer so log, EMF, local files, and experiment artifacts observe the same run rather than creating separate telemetry orchestration;
- add a service-side weighted resource governor so bounded per-query work remains bounded under concurrency;
- use a typed Go builder as the authoritative DSL and a restricted YAML layer for parameters and wiring, while keeping product algorithms in registered Go plugins;
- require every pipeline change to produce a plan report with data order, materialization points, asymptotic retained state, byte reservations, side effects, semantic identities, and crash/replay boundaries.

This approach preserves the strongest existing work—RagKit’s streamed canonical digests, staged backend builders, immutable bundle identity, schema-v2 content store, bounded verifier, and CoinVault’s authorization-before-fusion serving path—while removing the need to solve every new lifecycle or memory problem with another one-off command and diary-driven harness.

1. Scope, method, and evidence

1.1 Scope

This report examines the full CoinVault knowledge path:

1. MySQL and curated SQL-document inputs.

2. Source consistency and extraction.
3. Normalization, filtering, exclusions, and metadata.
4. Corpus identity and review artifacts.
5. Chunking and representation construction.
6. Embedding-cache lookup and provider work.
7. Vector and lexical index construction.
8. Bundle identity, verification, publication, and refresh.
9. Bundle opening and service startup.
10. Lexical/vector retrieval, authorization, fusion, reranking, and hydration.
11. `knowledge_search` registration and use by the CoinVault chat runtime.
12. Resource telemetry, capacity evidence, and operational workflow.

The objective is not only to reduce the current peak. It is to make future complex pipelines easier to express, review, extend, optimize, and operate under explicit memory limits.

1.2 Material analyzed

The analysis used:

- CoinVault source from the supplied GEC/CoinVault archive, including the current schema-v2 serving path, command composition, tool adapter, evaluation code, refresh and production design documents, and memory telemetry work.
- The connected goldeneagle/coinvault repository for the source connector implementation and the detailed COINVAULT-INDEX-00M-001 investigation diary, including commits and measurements not present as a complete package in the supplied archive.
- The supplied RagKit source, including BuildStream, the SQLite staging kernel, exact streamed JSON digesting, content SQLite, Blevé streamed construction and paged inspection, SQLite exact-vector construction/search, streaming verification, and bundle opening.
- The earlier self-optimizing RAG foundation report, to align this execution kernel with the proposed plan/compiler, plugin, evidence, and optimization planes.
- Primary references on external-memory algorithms, query iterator models, synchronous dataflow, MySQL consistent reads, SQLite atomicity/WAL, and Go's soft memory limit.

The uploaded CoinVault archive contains call sites for the latest internal/knowledgebuild APIs but not the full package implementation. The detailed diary, commit diffs, connected source files, current RagKit implementation, and current CoinVault serving code together expose the relevant algorithms and boundaries. Findings that depend on a historical implementation are labeled as such.

1.3 Terminology

Logical plan means the product-semantic graph: sources, transforms, relations, indexes, validators, and publication policy.

Physical plan means the chosen execution mechanisms: cursors, batches, joins, spills, storage adapters, schedules, reservations, and checkpoints.

Relation means a typed ordered collection with stable identity, schema, key, count, and canonical digest. It need not be SQL-facing; SQLite is one physical implementation.

Payload memory is memory occupied by document text, representation text, vector coordinates, decoded backend records, and similar large values.

Identity memory is compact state such as IDs, digests, ordinals, and relation keys.

Bounded always states the variable with respect to which a bound holds. "Streaming" alone is not a proof.

2. Current end-to-end system

2.1 Source boundary

CoinVault currently builds approved knowledge from three source classes:

Source	Main tables/assets	Indexed purpose	Explicitly excluded live facts
Product documents	products, metals, product_details	Product description, title, metal, facets, canonical URL	price, cost, inventory, orders, customer data
Category documents	categories	Public guide/category content	operational facts
Curated SQL documentation	embedded version-controlled SQL-doc library	Analyst-scoped schema guidance	live query results

The connector code correctly uses stable SQL ordering and stable document IDs. Product rows are ordered by p.id; facets are ordered by product_id, name, value; categories are ordered by c.id; SQL documentation topics and tables are sorted by name. Document IDs such as gec:product:<id> and gec:category:<id> are deterministic. Content digests are derived from normalized text. Access scope and source role are stored in metadata.

The main source-memory problem is not the SQL row scan itself. database/sql.Rows is cursor-shaped. The connector turns that stream into complete collections:

```
all product_details -> map[productID][]facet
all products        -> []Document
all categories       -> []Document
all SQL docs         -> []Document
all documents        -> concatenate + sort
```

For approximately 20,000 documents this is survivable, but it leaves the first half of the pipeline eager and makes memory proportional to corpus text. It also duplicates policy: source ordering exists in SQL, then the application sorts again.

2.2 Historical eager build

The original build path performed this sequence in process:

```
[]Document
-> corpus JSON buffer
-> optional furniture-stripped []Document
-> []Chunk
-> raw []Representation + breadcrumb []Representation
-> combined []Representation
-> batched provider calls, append all []Vector
-> eager indexbundle.Build
    -> backend canonical records
    -> Bleve construction
    -> SQLite exact-vector construction
```

The important defect was **overlapping liveness**. Embedding calls were batched, but the resulting vectors were accumulated. Batching provider requests bounded only one transient operation; it did not bound retained state.

At 114,106 representations and 1,536 float32 coordinates, coordinate storage alone is:

$$114,106 \times 1,536 \times 4 = 701,067,264 \text{ bytes}$$

before Go slice headers, per-vector objects, strings, maps, allocator fragmentation, decoded corpus objects, backend records, SQLite/Bleve state, and filesystem cache. The measured failure therefore matches the structural model.

2.3 Current staged build repair

The RagKit staged builder introduced a much better boundary:

```
type StreamInput struct {
    OutputRoot string
    CorpusPath string
    BatchSize  int
    Produce    func(context.Context, *Stager) error
    // identities and observers omitted
}

type Stager struct { /* private kernel */ }

func (*Stager) AddDocuments(context.Context, []rag.Document) error
func (*Stager) AddChunks(context.Context, []rag.Chunk) error
func (*Stager) AddRepresentations(context.Context, []rag.Representation) error
func (*Stager) AddVectors(context.Context, []rag.Vector) error
```

The stager:

- admits bounded batches synchronously;
- copies vector coordinates into compact little-endian float32 blobs;
- validates phase order, identities, lineage, dimensions, finite values, uniqueness, and predecessor existence;
- commits transaction-local writes before advancing counters or phase;
- seals an immutable build plan using canonical ordered scans;
- lets content, lexical, and vector backends consume ordered cursor producers;
- writes the manifest last and atomically renames the completed bundle directory.

The historical CoinVault patch, however, still constructed complete indexedDocuments, chunks, and reps slices, then called stageBatches over them. It eliminated the dominant Theta(ND) vector collection and backend overlap, but not the earlier Theta(total text payload) live set. The diary explicitly observed that the real build consumed more memory than the synthetic stager test because source text and representations were still eagerly materialized.

2.4 Bundle format and verification

RagKit schema v2 contains:

- a strict manifest;
- corpus and chunk/representation identities;
- a read-only content.sqlite store for documents and chunks;
- a Bleve lexical index;
- an optional SQLite exact-vector index;
- stable chunker, representation, provider/model/dimension, and backend identities.

The verifier now streams chunk and representation JSON arrays with a strict decoder, computes the same canonical JSON-array digest via digest.JSONSequence, retains compact identity maps, inspects Bleve in deterministic pages, and uses the vector backend's ordered streaming inspector. This preserves existing bundle IDs without retaining all text and backend hits.

The current `indexbundle.Open` calls the full streaming `Verify`, then opens read-only content, lexical, and vector handles. That is fail-closed and memory-safe at the measured corpus size, but it means a service restart performs release-grade verification work again. The architecture currently has no explicit distinction between:

- release verification before publication;
- receipt verification before activation;
- startup integrity verification;
- continuous/scheduled audit.

They can share validators without requiring the same physical plan each time.

2.5 Serving and tool path

The schema-v2 serving path is already close to the desired architecture:

```
coinvault serve
-> webchat/server composition root
-> resolve embedding profile if manifest has vectors
-> knowledge.OpenConfiguredTool
    -> indexbundle.Open
    -> read-only content.sqlite
    -> Bleve handle
    -> optional SQLite exact-vector handle
    -> serving mechanisms (fusion/reranker/synonyms/comparison plans)
    -> knowledge_search ToolEntry
-> register tool catalog
-> application-profile filtering
-> SessionStream/chat runtime
```

For a query, CoinVault:

1. validates and normalizes the request;
2. selects one or a small reviewed set of retrieval queries;
3. performs bounded lexical and optional vector search at `limit * searchDepth`;
4. collapses representation hits to chunk IDs;
5. fetches candidate metadata from `content.sqlite`;
6. applies access-scope and source-role authorization before cross-channel fusion or external reranking;
7. performs deterministic weighted reciprocal-rank fusion;
8. reranks only a configured bounded pool;
9. rechecks authorization;
10. hydrates only final bounded document/chunk evidence;
11. admits evidence into a run-scoped item/rune budget;
12. returns stable evidence labels to the model.

This is a strong memory and confidentiality shape. The exact SQLite vector backend scans all vectors but retains only the query vector and a top-k heap. Its time is approximately $\Theta(ND)$ per vector query, while its payload memory is $O(D + k)$. The dominant serving risk is therefore not one request; it is **multiplication by concurrent requests and optional reranker/provider work**, which is not currently expressed as one shared memory/cost reservation.

2.6 Operational and command surface

The single `cmd/coinvault/cmds/knowledge.go` file is approximately 2,433 lines and defines `build`, `smoke build`, `cache rebuild`, `package`, `cache export`, `bundle export`, `state inspect`, `seed`, `inspect`, `verify`, `evaluate`, `sweep`, and `judge` commands. Adjacent packages and tickets add:

- cache-only embedding runtime;
- cache archive validation;
- immutable bundle export;
- EFS state classification;
- memory readers and cgroup sampling;
- log and EMF sinks;
- local and container verifier harnesses;
- telemetry correlation and graph scripts;
- refresh run state, leases, publication, channel pointers, activation, and rollback designs;
- separate ECS task definitions, image inputs, capacities, roles, and mounted paths.

These capabilities are legitimate. Their *independent orchestration* is the accidental complexity. Most of them need the same primitives: open a generation, traverse an ordered relation, maintain a resource envelope, emit progress, verify identity, produce a receipt, and transition a durable state machine.

3. Evidence from the memory work

3.1 Failure and repair timeline

Experiment	Shape	Result	Important measurement
Original eager Fargate build	19,977 docs; 57,053 chunks; ~114k reps; 1,536 dims; 2 GiB	All embeddings completed; OOM before publish	exit 137
Local cache-only eager reproduction	same corpus until one missing cache item	Confirmed linear retained-vector growth	process peak ~1.456 GiB before backend
Original eager Fargate retry	same corpus; 8 GiB	All 114,106 embeddings completed; OOM immediately after input validation	last sampled RSS ~2.675 GiB; exit 137
Synthetic BuildStream shape test	19,977 / 57,053 / 114,106; batch 1,000	Pass	max RSS ~264 MiB
Real cache-backed BuildStream	exact corpus; 114,106 cache hits	Pass; immutable bundle published	Container Insights max ~1.125 GiB
Eager exact-bundle verifier in Docker	114,106 reps; 8 GiB limit	Pass but unsafe for 2 GiB	cgroup peak ~1.999 GiB
Streaming exact-bundle verifier	same bundle; 8 GiB	Pass	cgroup peak ~92 MiB
Streaming exact-bundle verifier	same bundle; 512 MiB limit	Pass	sampled cgroup peak ~87 MiB

3.2 What the measurements prove

The measurements establish five distinct facts.

First, the failure was not primarily a leak. It was a legal but incompatible live-set composition.

Second, provider batch size and worker count do not bound retained outputs. They control transient concurrency only.

Third, adding memory did not fix the algorithm. It delayed failure until a larger overlap boundary.

Fourth, a disk-backed relation plus ordered backend consumers removed dependence on vector-corpus payload size from the Go heap.

Fifth, build and verify are separate physical plans over the same logical data. Fixing one does not automatically fix the other; this is why the eager verifier remained multi-gibibyte after the builder succeeded.

3.3 What remains unproven

The current system does not yet provide a single formal bound from MySQL to tool response. In particular:

- product facets are retained for the entire source set;
- documents, chunks, and representations may still be complete slices before staging;
- inspect/statistics paths contain independent eager code;
- the writable staging database can reside under the bundle output root, which in deployment is EFS;
- one query is bounded, but aggregate service memory under concurrency is not enforced by a shared governor;
- release verification and startup verification are physically identical;
- capacity is inferred from measurement and configuration conventions rather than compiled from stage contracts;
- no plan linter rejects a newly added eager operator.

The next foundation should target these remaining properties rather than add another narrow memory patch.

4. The actual root cause: implicit liveness and duplicated life-cycle

4.1 Memory is a property of schedules, not just functions

A function that maps representations to vectors can be locally bounded while the overall program is unbounded. The relevant object is the **schedule and live-value graph**.

Let a pipeline be a directed acyclic graph $G = (V, E)$. Each edge carries values and each node may retain state. For a schedule σ , define:

- L_t : edges and node states live at time t ;
- $size(x)$: charged bytes of live value or reservation x ;
- X_t : external/native/page-cache memory charged to the process or cgroup;
- B : fixed runtime and executable baseline.

Then:

$$M_{peak}(G, \sigma) = \max_t \left(B + X_t + \sum_{x \in L_t} size(x) \right)$$

The eager implementation was expensive because the schedule kept documents, chunks, representations, and vectors live while backend construction introduced new values. BuildStream changed the schedule: payloads crossed a transaction boundary and became replayable disk relations; upstream slices could be released before each backend consumed the ordered relation.

This is analogous to register allocation and the red-blue pebble model of I/O complexity: the graph is not enough; placement and eviction decisions determine fast-memory pressure and data movement.

4.2 “Streaming” is underspecified

The term can describe very different guarantees:

Pattern	Retained state	Corpus-independent?	Example
Element streaming with append	Theta(N payload)	No	embed batches then append all vectors
Payload streaming with identity map	Theta(N IDs)	No, but compact	current verifier
Bounded batch streaming	O(batch * item size)	Yes for payload	Stager.AddVectors
Windowed/group streaming	O(max group size)	Yes if group is bounded	sorted product/facet merge
Bounded top-k	O(k + item scratch)	Yes	exact vector search
External sort	O(M) RAM; disk/I/O scale with N	Yes for memory	ordering unsorted source
Unbounded channel pipeline	depends on queue growth	No	producer outruns consumer

Every operator should therefore declare a retained-state class, not merely a boolean “streaming” label.

4.3 The current abstractions stop at the wrong boundaries

CoinVault/RagKit have good narrow APIs, but their boundaries are inconsistent:

- SQL connectors return slices.
- Chunkers generally operate on slices.
- Representations are composed as slices.
- Embedding APIs return batch slices.
- The new stager admits batches.
- Backend builders use producer callbacks.
- Verifiers use JSON streaming and backend-specific cursors.
- Serving uses index hits and bounded content lookups.
- Refresh uses artifact references and state transitions.

The abstractions improve as the data moves downstream, but there is no common relation/cursor contract connecting them. As a result, each layer reimplements ordering, counting, digesting, progress, cancellation, and failure context.

4.4 Product lifecycle is split into parallel state machines

There are currently several partially overlapping state machines:

Build stages:

```
input_validated -> staging_produced -> staging_sealed -> ... -> published
```

Verifier stages:

```
manifest -> chunks -> representations -> lexical -> vector -> complete
```

Open stages:

```
manifest -> verify stages -> indexes opened -> ready
```

Refresh states:

```
planned -> acquired -> extracting -> building -> verifying -> evaluating  
-> publishable -> published -> activating -> active / rejected / failed
```

Deployment states:

```
image built -> scanned -> task definition -> one-shot task -> bundle pointer  
-> service rollout -> health -> rollback
```

They should not be collapsed into one giant enum, but they should share one protocol:

```
Plan -> Run -> Checkpoint -> Seal -> Verify -> Receipt -> Publish -> Activate
```

A mode-specific stage is a child operation under that protocol. The current design instead places protocol logic in commands, package-specific callbacks, shell scripts, and Terraform procedures.

5. Mathematical and computer-science foundations

5.1 External-memory model

The Aggarwal-Vitter model distinguishes internal memory capacity M , block size B , and dataset size N . It treats block transfers, not arithmetic, as the scarce resource. For CoinVault, the exact disk model differs—EFS, local ephemeral storage, SQLite page caches, and provider I/O—but the design lesson is directly applicable:

- scans should be sequential and ordered;
- joins should exploit existing order where possible;
- sorts and random lookups should be explicit physical operators;
- a bounded RAM plan may deliberately spend more sequential I/O;
- the cost model must report both memory and I/O/latency.

A sequential relation scan takes approximately $\Theta(N/B)$ block transfers. External sorting requires approximately:

$$\Theta\left(\frac{N}{B} \log_{M/B} \frac{N}{B}\right)$$

I/Os. CoinVault should avoid external sorts when MySQL can provide stable key order and when transform order is preserved. When ordering cannot be proven, the compiler should insert an explicit external-sort node rather than allowing an application-level sort. Slice over all payloads.

5.2 Iterator/Volcano execution

The database Volcano model separates operators from the mechanism that requests the next record. A pull cursor has useful properties for this workload:

- the downstream consumer controls rate;
- backpressure is intrinsic;
- cancellation propagates along the call stack;
- no unbounded queues are required;
- ownership can be scoped to a call or batch;
- a materialization boundary is visible.

Pure record-at-a-time iteration can be expensive, so the recommended kernel is **vectorized pull**: a cursor fills a caller-bounded batch. The physical executor can choose batch capacity based on byte estimates and reservations.

5.3 Synchronous dataflow and static schedules

Synchronous dataflow is useful because actors declare production/consumption rates, permitting static schedules and finite buffer analysis. CoinVault is not fully synchronous: one document produces a variable number of chunks, cache misses are variable, and provider latency is asynchronous. Still, a restricted version is valuable:

- sources declare estimated and maximum record sizes;
- transforms declare upper bounds or “unknown” expansion rates;
- batches declare maximum items and bytes;
- asynchronous stages declare maximum in-flight work;
- materializers break cycles and variable-rate regions;
- the compiler computes a conservative schedule for memory-heavy stages.

The goal is not to import a DSP runtime. It is to borrow the idea that buffer capacity and firing policy are compile-time properties rather than accidental channel defaults.

5.4 Queueing theory and backpressure

Little’s Law states:

$$L = \lambda W$$

where L is average in-system work, λ throughput, and W residence time. If an embedding provider slows down while extraction continues, queued representation payload grows unless admission is bounded. Increasing concurrency can reduce W up to the provider/backend capacity, but also raises L , memory, and external cost.

Therefore every asynchronous operator must declare:

- maximum in-flight requests;
- maximum items/bytes per request;
- maximum queued batches;
- retry budget and backoff state;
- response-size bound;
- lease weight against the run budget.

A queue of “unlimited” is not an implementation default; it is a rejected plan.

5.5 Monoids and streaming identities

Canonical digests are naturally folds. If records are encoded canonically and order is fixed, an array digest can be computed incrementally while preserving the identity of an eager serialization:

```
hash("[")
for each record in order:
    hash(separator if needed)
    hash(canonical_json(record))
hash("]")
```

RagKit’s `digest.JSONSequence` already proves the key mechanism. The framework should generalize it as a relation identity service:

```
type RelationDigest[T any] interface {
    Add(T) error
    Sum() Digest
}
```

The identity must describe semantic records, not SQLite file bytes or incidental page layout. Physical stores can change without changing the relation digest.

5.6 Typestate and legal phase transitions

The current stager enforces phase order dynamically. The framework should retain runtime validation and expose typestate-like builders where practical:

```
EmptyGeneration
-> DocumentsWritten
-> ChunksWritten
-> RepresentationsWritten
-> VectorsWritten | LexicalOnly
-> SealedGeneration
-> VerifiedGeneration
-> PublishedGeneration
```

Go cannot express every transition statically without cumbersome generic types, so the external artifact and run status remain runtime-checked. The important design rule is that APIs do not accept ambiguous partially valid states. A publisher accepts `VerifiedGeneration`, not an arbitrary directory path.

5.7 Algebraic effects and ports/adapters

Pipeline semantics should remain independent of effects:

- MySQL read;
- embedding-provider call;
- local scratch write;
- bundle publication;
- channel activation;
- telemetry emission.

Each effect is a typed port implemented by an adapter. The logical plan declares required capabilities; the compiler and runtime bind concrete adapters. This allows the same plan to run with:

- a synthetic source;
- a frozen exported source;
- cache-only embeddings;
- a real provider;
- local filesystem publication;
- EFS/S3 publication;
- dry-run activation.

It also prevents product transforms from calling environment variables or AWS APIs directly.

6. A precise boundedness model

6.1 Boundedness lattice

Use the following ordered classification for every node and relation:

Class	Formal retained-state shape	Production interpretation
constant	$O(1)$	independent of data and configurable limits
dimension	$O(D)$	one embedding/query vector or model dimension scratch

Class	Formal retained-state shape	Production interpretation
batch	$O(B * S_{max})$	bounded batch of at most B items/bytes
window	$O(W_{max})$	one key group, heading section, or configured window
topk	$O(k)$ or $O(k * S_{max})$	ranked candidate heap/list
identity	$O(N * I)$	compact IDs/digests only
external	$O(M)$ RAM plus disk proportional to N	explicit spill/materialization
eager	$O(N * \text{payload})$	disallowed by default
unknown	no proved bound	compilation failure for production profile

The compiler may accept identity state for current cardinalities if a configured ceiling and estimate fit. It should report that this is not strict constant space. The streaming verifier is a good example: payload memory is bounded, but chunk and representation identity maps are cardinality-bounded.

6.2 Resource vectors, not one memory number

A node resource contract should be a vector:

$$R = (\text{heap}, \text{native}, \text{mmap}, \text{pagecache}, \text{scratch}, \text{goroutines}, \text{fds}, \text{providerCalls}, \text{tokens})$$

Some components cannot predict every field exactly. They should provide:

- a conservative reservation;
- a measured model by cardinality/dimension;
- an uncertainty margin;
- a hard maximum where enforceable;
- an admission key for shared resources.

For example:

```
resources:
  heap_bytes: 67108864
  scratch_bytes: 268435456
  max_in_flight: 2
  provider_calls: 1
  unknown_external_margin_bytes: 134217728
```

The run compiler aggregates these over the live schedule. The runtime verifies actual use and can cancel before the cgroup hard limit is approached.

6.3 Static live-set estimate

For a linear schedule with one active heavy stage at a time:

$$\widehat{M}_{peak} = M_{base} + \max_i (M_{node_i} + M_{input_i} + M_{output_i} + M_{backend_i}) + H$$

where H is safety headroom. For concurrent stages or fanout, sum reservations of overlapping nodes and edge buffers.

A compiler pass should produce a table such as:

Stage	Input live	Output live	Operator state	External margin	Estimated charged peak
product/facet merge	2 batches	1 document group	one facet group	driver buffers	32 MiB
chunk embed	1 document	1 chunk batch	heading parser	none	24 MiB
Bleve build	1 rep batch	1 vector batch	cache/provider	HTTP/native	192 MiB
	relation cursor	backend batch	index writer	Bleve/page cache	384 MiB
vector build	relation cursor	backend batch	SQLite writer	SQLite/page cache	256 MiB

The estimate is a review artifact and an admission input; measured cgroup maxima calibrate its margins.

6.4 Runtime leases

A shared weighted semaphore should enforce reservations:

```
type ResourceBudget interface {
    Acquire(ctx context.Context, request Reservation) (Lease, error)
}

type Lease interface {
    Reservation() Reservation
    Adjust(ctx context.Context, next Reservation) error
    Release()
}
```

The same mechanism applies to:

- build batches;
- concurrent provider calls;
- backend construction;
- service queries;
- reranker requests;
- evidence hydration;
- export compression/upload buffers.

Resource samples are observations attached to a lease. They do not replace the lease.

6.5 Go soft limit and cgroup hard limit

Go's GOMEMLIMIT covers memory managed by the Go runtime, not all cgroup-charged memory. SQLite, Bleve internals, memory mappings, kernel buffers, and page cache may be outside or only indirectly reflected. Therefore:

```
cgroup hard limit
> Go soft limit
> compiled maximum Go reservation
```

A reasonable initial policy is to set the Go soft limit to a measured fraction of the task limit after reserving explicit external margin. The exact percentage is workload-specific. The runtime limit protects against transient Go-heap overhead and encourages scavenging; it cannot turn an eager Theta(ND) algorithm into a bounded one and can produce GC thrashing if set below the live-set requirement.

7. Target architecture

7.1 Six layers

1. Product specification
CoinVault source policy, normalization, chunking, representations, embedding identity, retrieval policy, tool policy
2. Logical pipeline IR
typed nodes, ports, relations, invariants, semantic identities, effects, legal mutations
3. Physical compiler
cursor/batch/spill/join/sort/index strategy, schedule, reservations, checkpoints, storage placement, cost estimate
4. Execution kernel
run lifecycle, resource governor, cancellation, events, relation store, retries, resume, sealing, verification
5. Artifact and activation plane
immutable generations, receipts, publication, channels, CAS activation, rollback
6. Serving plane
verified read-only handles, query resource governor, bounded retrieval, authorization, fusion, reranking, hydration, tool catalog

This aligns with the earlier self-optimization architecture: the pipeline IR and compiler are the same optimizable program representation; the execution kernel is RagKit's data-plane foundation; ragopt can mutate and evaluate plan assets without owning product semantics.

7.2 One semantic generation, several physical views

A knowledge generation is not “a directory containing whatever the command wrote.” It is a semantic object:

```
type Generation struct {  
    ID          Digest  
    PlanID      Digest  
    SourceReceipt ArtifactRef  
    Relations   map[RelationName]RelationRef  
    Indexes     map[IndexName]IndexRef  
    Verification ArtifactRef  
    Publication *ArtifactRef  
}
```

Core relations:

```
document  
exclusion  
chunk  
representation  
vector  
source_receipt  
build_diagnostic
```

Physical stores may include:

<code>relations.sqlite</code>	canonical source/intermediate relations
<code>content.sqlite</code>	serving-optimized documents/chunks projection
<code>lexical/</code>	Bleve
<code>vectors.sqlite</code>	exact vector index
<code>manifest.json</code>	semantic and physical identities
<code>verification.json</code>	complete validator receipt
<code>publication.json</code>	object/file receipt, written last

`relations.sqlite` may be omitted from a minimal published serving bundle after all necessary canonical data has been projected and receipts are sufficient. During build and resume it is the single source of truth. Longer term, extending the final content store to carry canonical representation records can eliminate duplicated representation JSON and simplify audit/inspection.

7.3 Logical versus physical nodes

Logical node examples:

```
CoinVaultProducts
CoinVaultCategories
CuratedSQLDocs
NormalizeProduct
NormalizeCategory
FurniturePolicy
HeadingChunk
RawRepresentation
BreadcrumbRepresentation
EmbedRepresentation
LexicalIndex
VectorIndex
ContentProjection
BundleInvariantSuite
PublishGeneration
```

Physical implementations:

```
mysql.keyset_cursor
mysql.repeatabile_read_cursor
merge_join.sorted
map.batch
flatmap.document_local
sqlite.relation_writer
sqlite.ordered_cursor
embedding.cache_then_provider
bleve.stream_writer
sqlite_exact.stream_writer
json_sequence.digest
filesystem.receipt_publish
s3.multipart_publish
```

Optimization can replace a logical component or choose a different physical strategy without conflating the two identities.

7.4 Compiler passes

A production compiler should perform, in order:

1. **Schema/type checking** — ports match typed relations.

2. **Capability checking** — required source, provider, scratch, publication, and activation capabilities exist.
3. **Effect checking** — evaluation/dry-run plans cannot activate or publish unexpectedly.
4. **Order analysis** — determine which edges are ordered and insert explicit sort only where needed.
5. **Lineage analysis** — verify stable key and parent-child constraints.
6. **Bound analysis** — reject eager or unknown operators under a bounded profile.
7. **Liveness scheduling** — choose stage ordering and allowed overlap.
8. **Storage placement** — local scratch versus immutable output versus remote object store.
9. **Identity compilation** — canonical plan, source query set, transform, model, and backend IDs.
10. **Checkpoint plan** — define resumable materialization/receipt boundaries.
11. **Cost preflight** — estimate memory, scratch, I/O, provider calls, tokens, and duration.
12. **Lockfile emission** — freeze resolved plugin versions and physical choices.

The result is a canonical executable plan plus a human-readable explanation.

8. The end-to-end CoinVault physical plan

8.1 Snapshot and source receipt

A generation begins with one read-only MySQL transaction configured for REPEATABLE READ and a consistent snapshot. InnoDB consistent reads then share the snapshot established by the first read. All connector queries that participate in one generation should run through the same transaction handle.

```
type SnapshotSource interface {
    Begin(ctx context.Context, spec SnapshotSpec) (Snapshot, error)
}

type Snapshot interface {
    Cursor(ctx context.Context, relation SourceRelation) (RawCursor, error)
    Receipt(ctx context.Context) (SourceReceipt, error)
    Close() error
}
```

The receipt should include:

```
api_version: coinvault-source-receipt/v1
source: gec-mysql
server_identity: <reviewed non-secret identity>
database: gec_prod
isolation: repeatable-read
access_mode: read-only
consistent_snapshot: true
snapshot_started_at: ...
first_read_at: ...
completed_at: ...
query_set_digest: sha256:...
connector_version: coinvault-mysql/v3
counts:
  product_rows: ...
  product_detail_rows: ...
  category_rows: ...
  sql_doc_rows: ...
normalized_document_digest: sha256:...
exclusion_digest: sha256:...
```

Where available, record a GTID/binlog coordinate or managed-database snapshot identity as diagnos-

tic lineage, but do not make a nonportable value mandatory for semantic identity unless its stability is understood.

A long repeatable-read transaction can retain undo history on the database. The build should therefore separate **source capture** from expensive embedding and indexing:

```
short consistent MySQL snapshot
-> normalized local relation store
commit MySQL transaction
-> chunk/embed/index from local immutable relation
```

This both bounds database impact and makes retries provider-independent.

8.2 Streaming products and facets with a merge join

The current connector first loads every facet into a map. Both SQL queries are already sorted compatibly. Use a merge-group cursor:

```
products:      id = 1, 2, 3, 4, ...
product_details: product_id = 1,1,1, 3,3, 4, ...
```

Pseudocode:

```
func JoinProductsAndFacets(
    products Cursor[ProductRow],
    facets GroupCursor[int64, ProductFacet],
    emit func(ProductWithFacets) error,
) error {
    for products.Next() {
        p := products.Value()
        group, err := facets.TakeKey(p.ID)
        if err != nil { return err }
        if err := emit(ProductWithFacets{Product: p, Facets: group}); err != nil {
            return err
        }
        // group storage is released before the next product
    }
    return joinCompletenessCheck(facets)
}
```

Retained state becomes:

$$O(\text{size}(\text{product}) + \text{maxFacetsPerProduct})$$

rather than:

$$O(\text{totalFacetPayload} + \text{totalDocumentPayload})$$

The compiler can prove the join because both inputs declare `order_by product_id`. If a future connector cannot provide compatible order, it must request an external materialization/sort.

8.3 Normalization and exclusion as one pass

Normalization is a pure map/filter with two output relations:

```
raw source row/group
-> Normalize
+--> admitted Document
+--> Exclusion(reason, source ID, external ID)
```

Do not keep exclusions in a side slice. Write them to an ordered relation and digest them. This makes corpus shrinkage reviewable without retaining the admitted corpus.

The normalizer descriptor should identify:

- HTML-to-text version;
- whitespace collapse version;
- title fallback rules;
- minimum-rune policy;
- short-description deduplication rule;
- facet rendering order;
- metadata keys and scope policy;
- live-fact exclusion policy.

The document relation writer validates ID uniqueness, UTF-8, content digest, source role, access scopes, and canonical order in the same transaction that writes the row.

8.4 Document-local chunking

Heading-aware chunking does not require the full corpus. Its natural unit is one document:

```
type DocumentChunker interface {  
    Chunk(ctx context.Context, document rag.Document, emit func(rag.Chunk) error) error  
}
```

The physical operator holds one document, parser state, overlap tail, and one output batch. It writes chunks immediately. It validates:

- document existence;
- byte/rune ranges;
- nonnegative and unique per-document ordinal;
- content digest;
- chunker identity;
- stable chunk ID.

A document whose text itself exceeds the permitted per-item bound should either be rejected with an attributable exclusion/failure or spooled to a large-object path and parsed by a specialized bounded reader. Silent allocation of an arbitrarily large document violates the plan.

8.5 Chunk-local representations

Raw and breadcrumb representations can be produced from one chunk plus compact document metadata. The relation query should join chunk and document by key without loading all documents:

```
ordered chunk cursor  
+ point/merge access to document title/metadata  
-> raw representation  
-> breadcrumb representation
```

The preferred physical strategy is an ordered SQL join inside the relation store:

```
SELECT ...  
FROM chunk c  
JOIN document d ON d.id = c.document_id  
ORDER BY c.ordinal, representation_kind;
```

or equivalent application cursor with one-document cache. This makes representation order explicit and avoids two full representation slices followed by Compose.

8.6 Embedding as a bounded asynchronous island

Embedding is the one naturally asynchronous stage. It should be isolated between two durable relations:

```
sealed representation relation
  -> bounded batch reader
  -> cache lookup
  -> bounded provider requests for misses
  -> validate dimensions/finite values/identity
  -> vector relation transaction
```

The stage owns:

- at most `batch_items` representations;
- at most `max_in_flight` request batches;
- at most `batch_items * dimensions * 4` vector bytes per response batch;
- bounded retry/backoff state;
- no accumulated vector slice.

Cache semantics must include all output-determining identity:

```
canonical representation bytes
representation kind/transform identity
provider
model
embedding dimensions
provider-specific document prefix/version
normalization version
```

A cache hit is admitted through the same validator as a provider result. Cache storage is an adapter, not a special bypass around relation invariants.

For deterministic replay, commit each vector batch before advancing the durable checkpoint. On resume, the vector relation's primary key and embedding identity decide what remains. Provider accounting is reconstructed from events and cache metadata, not from guessed loop indexes.

8.7 Seal before backend construction

Once all semantic relations pass validation, seal the generation plan:

```
document digest/count
exclusion digest/count
chunk digest/count
representation digest/count/kinds
vector digest/count/dimensions (if enabled)
source receipt digest
logical plan ID
physical lockfile ID
```

After sealing:

- semantic relations become read-only;
- no source/provider work is allowed;
- backend creation is a deterministic projection;
- identical semantic generations may reuse a verified existing bundle;
- failures can resume at backend boundaries without reopening MySQL or calling the provider.

8.8 Build backends sequentially from the same relations

For a one-shot build, do not fan one stream concurrently to content, Bleve, and vector writers. Concurrent fanout requires queues and holds the slowest consumer's lag. Execute deterministic projections sequentially:

```
sealed relation store
-> build content.sqlite -> verify identity -> checkpoint
-> build Bleve          -> verify identity -> checkpoint
-> build vectors.sqlite -> verify identity -> checkpoint
```

This increases relation scans but minimizes overlapping native memory and simplifies resume. The external-memory model favors sequential scans, and the measured failure history justifies the trade.

Each backend plugin receives an ordered producer and an expected count/digest. It must report its own content identity and resource samples. It cannot read arbitrary bundle files or mutate other projections.

8.9 Local scratch, immutable publication, and EFS

Writable SQLite staging should live on local ephemeral storage. SQLite's own documentation warns that network filesystem locking can be unreliable and that WAL shared-memory semantics do not work over network filesystems. Even when one writer is expected, using EFS for active SQLite transactions adds correctness and latency uncertainty.

Recommended storage topology:

```
Fargate ephemeral local storage
/scratch/run-<id>/relations.sqlite
/scratch/run-<id>/bundle/

EFS or S3 immutable publication area
/knowledge/incoming/<bundle-id>-<run-id>/
/knowledge/bundles/<bundle-id>/
```

Publication procedure:

1. Build and verify all files locally.
2. Create an immutable file receipt with relative paths, sizes, and SHA-256 digests.
3. Copy into a unique remote incoming directory/object prefix using bounded buffers.
4. Re-read or provider-check every copied object's size/digest.
5. Write `publication.json` last.
6. On EFS, rename the completed incoming directory to the final bundle path within the same filesystem, or treat the receipt marker as the visibility boundary when rename semantics are not used.
7. Never reopen a published file writable.

The final read-only SQLite files may be served from EFS after all writer handles are closed. Publication is not the same operation as the staging transaction.

8.10 Verification profiles

Use one validator registry with several compiled profiles:

Profile	Purpose	Work
build-seal	Before local bundle completion	relation counts/digests, lineage, backend identity

Profile	Purpose	Work
release-full	Before publication/promotion	complete streaming relation and backend inspection, security policy, retrieval fixtures
startup-receipt	Service restart	strict manifest/receipt, expected file identity, read-only open, backend manifests, optional sampled fixtures
audit-full	Scheduled/incident	same or stronger than release, independent environment
debug-targeted	Operator diagnosis	selected relation/index validators

A production policy may continue full streaming verification at startup if its latency is acceptable. The key change is to represent that choice in the plan, not hard-code `Open == full Verify` forever. A signed or otherwise custody-protected release receipt can permit a faster startup without weakening the pre-promotion gate.

8.11 Activation and rollback

Activation consumes only a verified, published generation:

```
type Activator interface {
    CompareAndSwap(ctx context.Context, expected Channel, next PublishedGeneration)
    ⇨ (Channel, error)
}
```

The channel pointer records generation, previous bundle, decision/verification receipt, image/runtime compatibility, and update time. Service rollout verifies that its reported active bundle and runtime identities match the release tuple. Rollback switches the pointer back and repeats health checks; it never deletes the failed candidate.

9. Serving as a resource-bounded pipeline

9.1 Current query plan

The current serving algorithm is logically sound:

```
query normalization / reviewed decomposition
|
+--> lexical top-L --collapse--> authorize metadata --+
|
+--> vector top-L --collapse--> authorize metadata --+--> weighted RRF
                                                                -> optional rerank
                                                                -> final authorize
                                                                -> hydrate top-k
                                                                -> evidence ledger
```

The proposal is to describe this as a query plan using the same operator descriptors and resource contracts as the build. Build and query modes differ in their physical profile, but not in the vocabulary of typed relations, effects, identity, and reservations.

9.2 Per-query bound

A conservative per-query memory model is:

$$M_q = M_{queryVector} + M_{lexicalHits} + M_{vectorHeap} + M_{candidateMetadata} + M_{rerankPool} + M_{hydratedEvidence} + M_{provider}$$

With fixed dimensions, retrieval depth, reranker pool, maximum chunk bytes, and evidence rune budget, this is parameter-bounded.

The runtime should derive a Reservation before execution:

```
type QueryReservation struct {  
    CandidateItems int  
    CandidateBytes int64  
    VectorDims     int  
    RerankerItems  int  
    EvidenceRunes  int  
    ProviderCalls  int  
}
```

A service-wide governor limits the sum of active reservations. This prevents C simultaneous individually bounded queries from exceeding the task envelope:

$$M_{service} \leq M_{base} + \sum_{q \in active} M_q + H$$

When the governor cannot admit a request, the service should queue within a small timeout or return a clear overload response. It should not let arbitrary goroutine concurrency delegate admission to the cgroup OOM killer.

9.3 Exact vector search

The SQLite exact-vector backend is a useful baseline:

- memory: $O(D + k)$ plus database/page-cache effects;
- time: $O(ND)$;
- deterministic ordering with stable tie-breaking;
- no external ANN dependency;
- simple content identity and verification.

This is acceptable at approximately 114k representations if measured latency and query concurrency meet requirements. The planner should expose its cost model. When growth or traffic makes it inadequate, an ANN backend is a plugin with its own build, verification, recall, memory, and serving contracts—not a rewrite of the kernel.

9.4 Authorization as a relational operator

Authorization-before-fusion is not incidental application code. It is an effect-free predicate over candidate metadata with a trusted scope input. Model it as a mandatory operator:

```
CandidateID -> CandidateMetadata -> AuthorizedCandidate
```

The type system should prevent an UnscopedCandidate from entering external reranking or hydration in a protected plan. This is a useful use of semantic port types:

```
type UnscopedCandidate struct { ... }  
type AuthorizedCandidate struct { ... }
```

A final authorization pass remains defense in depth.

9.5 Reranking and provider boundaries

Reranking has both memory and data-governance effects. Its descriptor declares:

- maximum pool items and bytes;
- text format and truncation policy;
- model/provider/adaptor identity;
- outbound-data classification;
- timeout/retry/call budget;
- whether scores are deterministic enough for cache/replay;
- native memory margin if local.

The compiler rejects a plan that can send unauthorized or unbounded candidate text to a reranker.

9.6 Hydration and evidence

Content hydration should remain a bounded final projection from IDs to source documents/chunks. `content.sqlite` is the correct serving abstraction. The evidence ledger's item and rune limits should become a generic `BudgetedCollect` operator, with explicit omission accounting.

The tool output already exposes bundle, query-transform, retrieval-policy, evidence-ledger, reranker, description, and effective-limit identities. Preserve these as the query-run receipt. Add the compiled query-plan ID and resource reservation/actuals so evaluation and production traces use the same semantics.

10. The compositional kernel

10.1 Minimal Go contracts

The kernel should be small enough to understand in one sitting.

```
type Batch[T any] struct {
    Values []T
    Bytes  int64
    lease  Lease
}

// Values are valid until the next call or Close. Consumers must copy only
// what they intentionally retain under a separate reservation.
type Cursor[T any] interface {
    Next(ctx context.Context, dst []T) (n int, err error)
    Close() error
    Descriptor() RelationDescriptor
}

type Source[T any] interface {
    Open(ctx context.Context, run RunContext) (Cursor[T], error)
}

type Transform[I, O any] interface {
    Apply(ctx context.Context, input I, emit func(O) error) error
    Descriptor() OperatorDescriptor
}

type Sink[T any] interface {
```

```

    Consume(ctx context.Context, source Cursor[T], run RunContext) (ArtifactRef, error)
    Descriptor() OperatorDescriptor
}

```

The callback-style producer already used in RagKit remains valid for backend adapters. The kernel can bridge cursor and producer without making either public style universal.

10.2 Ownership rule

The most important API rule is:

A value belongs to the current cursor/batch lease. Retaining it after the call requires an explicit copy under another lease or materialization into a relation/artifact.

This makes accidental append-based accumulation visible in review. Helper APIs should avoid returning naked slices for corpus-scale operations.

10.3 Operator descriptor

```

type OperatorDescriptor struct {
    Kind          string
    Version       string
    InputSchema   SchemaRef
    OutputSchema  SchemaRef
    Deterministic bool
    Order         OrderContract
    Expansion     ExpansionBound
    Retained      StateBound
    Effects       []Effect
    Restart       RestartContract
    CostModel     CostModelRef
}

```

StateBound encodes the boundedness lattice. ExpansionBound may be exact, maximum-per-input, or unknown. Production compilation rejects unknown output size when it would feed a memory-resident collector.

10.4 Relation descriptor

```

type RelationDescriptor struct {
    Name          string
    Schema        SchemaRef
    PrimaryKey    []string
    Order         []OrderKey
    Count         int64
    Digest        Digest
    Storage       StorageRef
    Classification DataClassification
}

```

Relations are immutable after seal. A writer may expose a checkpoint count and digest prefix while active, but readers outside the owning run see only sealed relations.

10.5 Materialization is explicit

A materializer is not a hidden implementation detail. It is a node:

```
Cursor[T] -> Materialize(sqlite, key/order/indexes) -> RelationRef[T]
```

Reasons include:

- source snapshot closure;
- variable-rate boundary;
- replay/resume;
- external sort or join;
- multiple downstream consumers;
- provider isolation;
- backend sequentialization.

Every materialization reports disk estimate, transaction policy, indexes, and cleanup/retention policy.

10.6 No implicit goroutine graph

Do not implement every edge as a goroutine and channel. For this workload, the default executor should be synchronous pull with explicit asynchronous islands. Benefits:

- natural backpressure;
- simpler failure propagation;
- fewer hidden buffers;
- deterministic schedules;
- easy resource lease accounting;
- easier crash/replay reasoning.

A plugin may use bounded internal concurrency, but its descriptor and lease include all in-flight work.

10.7 Unified run events

One event vocabulary should replace mode-specific observation plumbing:

```
type Event struct {
    RunID      string
    Sequence   uint64
    Time       time.Time
    NodeID     string
    Attempt    int
    Kind       EventKind
    Progress   *Progress
    Resources  *ResourceSample
    Artifact   *ArtifactRef
    Checkpoint *CheckpointRef
    Failure    *Failure
}
```

Stable kinds:

```
run_started
node_started
progress
resource_sample
checkpoint_committed
artifact_sealed
node_completed
node_failed
```

```
run_completed
run_failed
```

Adapters render this stream to JSONL, Zerolog, CloudWatch EMF, ragopt artifacts, or an operator UI. Build and verify do not need separate sink orchestration.

10.8 Checkpoint and resume

A checkpoint is legal only at a durable idempotent boundary:

```
type CheckpointRef struct {
    NodeID      string
    InputDigest Digest
    OutputPrefix Digest
    CursorKey   []byte
    Artifact    ArtifactRef
    SemanticState string
}
```

Resume recompiles the same plan/lockfile, verifies source and preceding artifacts, and starts from the next valid checkpoint. It never resumes from a guessed in-memory index.

11. A practical DSL

11.1 Go builder is authoritative

The first DSL should be a typed Go builder because source transforms are product code and need compiler support, tests, and refactoring. Example:

```
plan := pipeline.New("coinvault-knowledge/v3").
    Budget(pipeline.Memory("768MiB"), pipeline.Scratch("20GiB")).
    Source("products", mysql.Products()).
    Source("facets", mysql.ProductFacets()).
    MergeJoin("product_with_facets", "products", "facets", ByProductID()).
    Transform("documents", coinvault.NormalizeProducts()).
    Union("documents", coinvault.Categories(), coinvault.SQLDocs()).
    Materialize("documents", relation.SQLite()).
    FlatMap("chunks", ragkit.HeadingChunker(chunkSpec)).
    FlatMap("representations", ragkit.Representations(repSpec)).
    Materialize("representations", relation.SQLite()).
    Transform("vectors", ragkit.Embed(embedSpec)).
    Materialize("vectors", relation.SQLite()).
    Sink("content", ragkit.ContentSQLite()).
    Sink("lexical", ragkit.Bleve(lexicalSpec)).
    Sink("vector", ragkit.SQLiteExact(vectorSpec)).
    Validate("release", coinvault.ReleaseValidators()).
    Publish("bundle", artifact.ImmutableBundle()).
    Build()
```

The builder emits the serializable logical IR and is not itself the runtime.

11.2 Restricted YAML authoring layer

YAML should configure registered components and bounds, not contain arbitrary expressions:

```
api_version: rag-pipeline/v1
name: coinvault-openai-small
```

```

profile: bounded-production
budget:
  memory: 768MiB
  scratch: 20GiB
  provider_calls: 120000

sources:
- id: products
  use: coinvault.mysql.products/v3
  config:
    min_description_runes: 80
- id: facets
  use: coinvault.mysql.product_facets/v2

nodes:
- id: product_join
  use: core.merge_join/v1
  inputs: [products, facets]
  config: {left_key: id, right_key: product_id}
- id: documents
  use: coinvault.normalize_products/v3
  inputs: [product_join]
- id: chunks
  use: ragkit.heading_chunk/v2
  inputs: [documents]
  config:
    maximum_runes: 1600
    overlap_runes: 160
- id: representations
  use: ragkit.raw_breadcrumb/v2
  inputs: [chunks]
- id: vectors
  use: ragkit.embed/v2
  inputs: [representations]
  config:
    profile: openai-small
    batch_items: 1000
    max_in_flight: 2

materialize:
- relation: documents
  using: sqlite
- relation: representations
  using: sqlite
- relation: vectors
  using: sqlite

sinks:
- use: ragkit.content_sqlite/v1
- use: ragkit.bleve/v2
- use: ragkit.sqlite_exact/v1

verify:
  profile: coinvault.release/v2
publish:
  use: coinvault.immutable_generation/v1

```


The compiler resolves component versions and emits a lockfile. The YAML cannot introduce a loop, shell command, arbitrary SQL, or unknown plugin without registry policy.

11.3 Physical plan report

coinvault knowledge plan should emit:

```
Plan ID: ...
Source consistency: repeatable-read read-only snapshot
Materializations: documents, representations, vectors
Writable scratch: local ephemeral
Publication: EFS receipt-last

Node                State bound      Reservation      Order
products/facets merge window          24 MiB          product_id
normalize            batch           16 MiB          document_id
chunk                document-local  32 MiB          document_id,ordinal
represent            batch           32 MiB          chunk_id,kind
embed                batch+inflight  192 MiB         representation_id
content projection   external        192 MiB         document/chunk order
Bleve build          external        384 MiB         representation_id
vector build         external        256 MiB         representation_id

Estimated charged peak: 640 MiB
Required task limit with margin: 1 GiB
Unknown bounds: none
Eager nodes: none
External effects: MySQL read, embedding provider, cache write, EFS publish
Activation capability: absent
```

This is the artifact reviewers currently reconstruct from code, telemetry, and diary entries.

12. Plugin and package architecture

12.1 Repository ownership

Recommended split:

```
ragkit/
  pipeline/          generic typed IR and compiler contracts
  execution/         cursor, batch, budget, events, checkpoints
  relation/          canonical relation and SQLite implementation
  artifact/          immutable generation and receipt primitives
  rag/operators/     chunk, representation, embedding adapters
  rag/index/...      content, Bleve, vector sink plugins

ragopt/
  study/             plan assets and candidate patches
  executor/          external run integration
  evaluation/        compare/gate/diagnostics over run receipts

coinvault/
  internal/knowledgeplan/ product logical plan and defaults
  internal/mysqlsource/  consistent snapshot and typed source cursors
  internal/knowledgenorm/ product/category/SQL-doc transforms
  internal/knowledgepolicy/ scope/live-fact/source policy validators
  internal/knowledgeserve/ product query plan and tool projection
```

internal/knowledgeops/ cmd/coinvault/...	publication/channel/activation adapters thin command bindings
---	--

The current internal/knowledgebuild should shrink into product adapters and plan construction. Generic staging, telemetry, resume, and artifact custody belong below it.

12.2 Plugin classes

Minimal registry classes:

```
SourceFactory
TransformFactory
RelationStoreFactory
IndexSinkFactory
ValidatorFactory
PublisherFactory
ActivatorFactory
SearchOperatorFactory
CostModelFactory
```

Each factory returns a descriptor and implementation. Descriptors are serializable and versioned. The kernel never switches on product-specific enum values.

12.3 Extension tiers

Start with statically linked Go registrations. They provide type safety, debugging, and predictable deployment. Add subprocess/WASI plugin tiers only when independent deployment or untrusted extension is a demonstrated need. Dynamic Go .so plugins should not be the public compatibility contract.

12.4 Conformance suites

Every plugin class receives a reusable conformance suite:

- cancellation and deadline propagation;
- deterministic output under repeat;
- order contract;
- ownership/no-retention probes where measurable;
- strict schema and unknown-field rejection;
- bounded batch enforcement;
- crash before/after checkpoint;
- digest parity with reference implementation;
- malformed input and duplicate-key rejection;
- resource event completeness;
- no unauthorized effects in dry-run/evaluation profile.

13. Identity model

13.1 Separate identities

Use at least these identities:

Identity	Meaning
Source query-set ID	SQL/assets and connector semantics
Source receipt ID	Exact observed snapshot counts/digests

Identity	Meaning
Normalization ID	Output-determining text/metadata policy
Chunker ID	Chunking algorithm and parameters
Representation ID	Representation transforms/prefixes
Embedding ID	provider/model/dimensions/document transform
Logical plan ID	Product-semantic graph
Physical plan ID	schedule, batch, spill, backend strategy
Relation IDs	canonical ordered semantic records
Bundle/generation ID	semantic relation + backend identities
Verification receipt ID	validators, versions, results, artifacts
Publication receipt ID	exact published bytes and location
Query plan ID	query transforms, channels, auth, fusion, rerank, hydration
Runtime release ID	image + bundle + profiles + tool policy

This prevents execution tuning—batch size, workers, scratch path—from silently changing semantic cache keys while still allowing physical plans to be compared operationally.

13.2 Canonical record encoding

Relation identity should be based on a versioned canonical encoding. Continue the existing JSON-sequence parity where compatibility is required. For new relations, define field order through structs/schema, normalize nil versus empty consistently, reject non-finite values, and make order part of the descriptor.

Physical database identity includes its semantic content digest and backend version, not a hash of potentially nondeterministic file pages alone. A publication receipt additionally hashes exact bytes for transfer integrity.

13.3 Plan lockfile

Example:

```
{
  "api_version": "rag-pipeline-lock/v1",
  "logical_plan_id": "sha256:...",
  "physical_plan_id": "sha256:...",
  "components": {
    "coinvault.mysql.products": "v3@commit:...",
    "ragkit.heading_chunk": "v2@commit:...",
    "ragkit.bleve": "v2@commit:..."
  },
  "schemas": {"document": "rag-document/v1"},
  "orders": {"document": ["id"]},
  "resources": {"memory_bytes": 805306368},
  "storage": {"scratch": "local-ephemeral", "publish": "efs"}
}
```

Resume and ragopt comparison require the same lockfile unless the candidate explicitly mutates an allowed field.

14. Verification as compiled invariants

14.1 Invariant classes

Validators should be registered under five classes:

1. **Structural** — schema, counts, unique keys, foreign keys, order, dimensions, finite numbers.
2. **Semantic** — content digests, raw representation equals chunk content, bundle ID, transform identities.
3. **Backend** — content/Bleve/vector manifests and content digests; open/read-only behavior.
4. **Product policy** — roles/scopes, live-fact exclusion, canonical URL/source constraints, privacy classification.
5. **Behavioral** — protected retrieval fixtures, authorization cases, latency/resource ceilings.

Each validator declares required relations/indexes and retained-state bound. The compiler schedules them over existing readers. Verification no longer needs a bespoke traversal for every command.

14.2 Differential validation

For every conversion from eager to bounded execution, use differential tests:

```
same frozen input
-> reference eager output/digests
-> bounded output/digests
-> byte/semantic equality where promised
```

This is how the current streaming verifier correctly preserved schema-v1 digests. Keep the reference path only in tests or small-fixture tooling after production migration.

14.3 Fail closed on incomplete evidence

A release receipt should state every validator and result. Missing validators are not passes. Unknown plugin versions, partial files, omitted counts, skipped protected fixtures, or telemetry failure at a required boundary produce hold or fail according to policy.

15. Observability without a parallel architecture

15.1 One event stream

The current memory instrumentation is useful and should remain, but it should become an adapter over run events. Logical progress, resource samples, cache counters, provider counters, GC, cgroup values, artifacts, and terminal state belong in one ordered JSONL authority.

CloudWatch EMF can project bounded numeric fields with stable low-cardinality dimensions. Human-readable logs can project the same events. The command's final Glazed table should use a separate output stream or explicit framing so event parsers do not need to guess where JSON ends.

15.2 Measurement hierarchy

Use three levels:

- **reservation**: what the planner/runtime admits;
- **runtime metrics**: Go heap/system, node-owned bytes, queue/in-flight counts;
- **container truth**: cgroup current/peak, anonymous/file, OOM state.

An alert should compare actual to reservation and task limit. A node that repeatedly exceeds its model is a cost-model defect even when the task does not OOM.

15.3 Cardinality and stage correlation

Every progress event should carry relation position:

```
input relation/count or key
output relation/count
batch items/bytes
cache hits/misses/provider calls
checkpoint sequence
```

This makes capacity models reusable without custom timestamp-join scripts. External samplers remain valuable for cgroup attribution, but the run event already identifies the current node and lease.

15.4 Privacy

Preserve the current discipline: metrics contain counts, timings, digests, bounded stable component names, and terminal classes—not document text, SQL values, credentials, customer data, arbitrary bundle paths, or run IDs as metric dimensions.

16. Engineering workflow

16.1 Plan-first change process

Every pipeline change follows:

```
edit logical plan/component
-> compile plan
-> review diff of semantics, effects, bounds, identities
-> fast conformance tests
-> differential small fixture
-> hard-limit shape test
-> real frozen snapshot canary
-> release verification
-> ragopt/evaluation gate when semantics changed
-> publication/promotion plan
```

The plan diff should answer:

- what data changes;
- what execution only changes;
- which relations rebuild;
- which caches remain valid;
- which resource bound changes;
- which effects/capabilities are newly required;
- which validators and evaluation suites run.

16.2 CI tiers

Fast PR checks

- IR/schema and plugin registry validation.
- No unknown/eager nodes in production profile.
- Unit and conformance tests.
- Digest/order golden tests.
- `go test -race` for kernel/adapters.
- generated plan/lockfile cleanliness.

Bounded integration checks

- synthetic geometry under a hard cgroup limit;
- one cache-hit and one cache-miss embedding run with a fake provider;
- crash injection at every checkpoint;
- local scratch exhaustion;
- malformed/duplicate/out-of-order source data;
- exact verify/open/query path.

Scheduled provider-backed checks

- frozen approved snapshot;
- exact model/profile identity;
- provider budgets and retry accounting;
- build and query resource envelope;
- retrieval and answer evaluation where required.

Release checks

- independent full verification;
- publication receipt reconstruction;
- read-only bundle mount;
- service startup profile;
- concurrent query load under task limit;
- activation and rollback drills.

16.3 Memory regression tests

A memory test is successful only when it states:

```
input geometry
physical plan ID
container hard limit
Go soft limit
CPU limit
scratch placement
cache/provider mode
peak cgroup current/anon/file
peak Go heap/system
runtime and throughput
terminal identity
```

Use both asymptotic shape sweeps and exact production-shaped fixtures. A lower final RSS is irrelevant if the transient peak exceeds the task limit.

16.4 Crash testing

Inject failures:

- before and after each SQLite transaction commit;
- before/after relation seal;
- during backend construction;
- after file copy but before publication receipt;
- after channel CAS but before service health;
- during rollback.

The acceptance property is not “no partial files exist.” Partial run-scoped artifacts may exist. The property is that no partial artifact is observable as a sealed/published/active generation and that resume/reconciliation is deterministic.

16.5 Documentation workflow

The detailed diaries have been valuable for discovery, but stable architecture facts should graduate into:

- versioned descriptors;
- plan reports;
- ADRs;
- conformance tests;
- run receipts;
- generated source maps;
- capacity dashboards.

A future engineer should not need to read thirty diary steps to learn that vectors must not be accumulated or that writable staging must use local scratch.

17. Integration with self-optimization

The execution kernel is a prerequisite for safe optimization across indexing dimensions. A ragopt candidate can patch:

- source inclusion policy;
- normalization/furniture policy;
- chunker parameters;
- representation kinds/prefixes;
- embedding model/dimensions;
- lexical backend/boosts;
- vector backend/configuration;
- retrieval depth/fusion/reranker;
- tool/evidence policy;
- physical batch/concurrency/scratch strategy.

The compiler classifies each patch:

```
semantic build change -> rebuild affected relations/indexes
semantic query change -> reuse bundle; rebuild query plan only
physical-only change  -> same semantic generation; capacity benchmark
policy/evaluator change -> no candidate comparison without explicit study change
```

Content-addressed relations and explicit dependencies enable common-subexpression reuse. For example, a reranker change reuses all index artifacts; a vector-backend change reuses documents/chunks/representations/vectors; a chunker change invalidates descendants but not the source snapshot.

Resource objectives become first-class evaluation metrics:

- peak cgroup memory;
- scratch bytes;
- build I/O and duration;
- provider calls/tokens;
- service query memory and latency under concurrency;
- startup verification duration;
- retrieval/answer quality.

Promotion remains constrained multi-objective selection, not “lowest memory wins.”

18. Migration plan

18.1 Phase 0 — Freeze contracts and delete ambiguity

Deliverables:

- inventory every knowledge command and mode;
- define the boundedness lattice and resource vector;
- freeze current semantic identities and exact bundle compatibility requirements;
- define canonical run event, relation descriptor, generation, verification, and publication receipt schemas;
- identify which current scripts are experiments versus permanent operations.

Exit condition: one architecture map explains every command in terms of shared lifecycle operations.

18.2 Phase 1 — Minimal kernel around existing RagKit

Implement:

- Cursor, Batch, ownership documentation;
- RelationDescriptor and SQLite relation store;
- resource reservations and run events;
- logical/physical plan descriptors;
- adapters around existing BuildStream, verifier, content, Blevé, and vector builders.

Do not rewrite the staged kernel initially. Wrap and generalize proven mechanisms.

Exit condition: current staged build and verify execute through a plan and emit one run journal without changing bundle identity.

18.3 Phase 2 — Stream MySQL into the document relation

Implement:

- one read-only repeatable-read snapshot;
- typed product, facet, category cursors;
- sorted merge join for product/facets;
- streaming normalization and exclusions;
- local relation materialization;
- source receipt and resume after snapshot capture.

Exit condition: peak payload memory during extraction is independent of total corpus text; document digest matches the current reference build.

18.4 Phase 3 — Remove eager chunks and representations

Implement per-document chunking and per-chunk representation writers. Replace full-slice composition with relation scans. Preserve exact IDs, order, and digests.

Exit condition: no production path constructs []Document, []Chunk, or []Representation proportional to the full corpus before staging.

18.5 Phase 4 — Consolidate generation storage

Use the relation store as the source for content, lexical, vector, verification, inspect, statistics, and package/export. Decide whether representations move into an extended canonical SQLite store and whether legacy JSON files remain compatibility projections.

Exit condition: all bundle modes traverse common relation readers; duplicate eager inspect/statistics loaders are retired.

18.6 Phase 5 — Correct storage placement and publication

Move writable staging to local ephemeral storage. Implement bounded receipt-last publication to EFS/S3 and reconciliation of incoming prefixes. Add immutable/read-only assertions.

Exit condition: no active SQLite writer uses EFS; interrupted copy cannot become publishable.

18.7 Phase 6 — Service resource governor

Compile the query plan, estimate per-query reservation, and admit lexical/vector/rerank/hydration work through one weighted governor. Add load tests under the actual ECS hard limit.

Exit condition: the service has a documented and enforced concurrency/memory envelope, not just bounded individual functions.

18.8 Phase 7 — Unify refresh and promotion

Implement refresh as a coordinator over generation runs and receipts. Reuse the same event/run store, publication adapter, validators, and channel CAS. Keep scheduling in EventBridge/Batch or River; do not turn the kernel into a scheduler service.

Exit condition: build, inspect, verify, publish, activate, and rollback commands are thin views over one run protocol.

18.9 Phase 8 — Remove legacy paths

Delete or quarantine:

- eager production builders;
- duplicate verifier/open/inspect traversals;
- command-specific telemetry loops;
- cache/bundle export implementations that bypass artifact interfaces;
- ad hoc newest-bundle activation;
- stale JSON compatibility data after a planned schema migration.

Exit condition: production compilation contains no eager or unknown node and the repository no longer has two authoritative lifecycle implementations.

19. Concrete first vertical slice

The best first slice is **MySQL snapshot to sealed document relation**. It is upstream of all current staged improvements and proves the new kernel without changing indexes or serving.

Implement:

```
coinvault knowledge plan
coinvault knowledge capture
```

capture should:

1. compile a plan with a 256 MiB memory reservation and local scratch;
2. begin one read-only repeatable-read MySQL snapshot;
3. stream products and facets through the merge join;
4. stream categories and SQL docs;
5. normalize and write documents/exclusions to `relations.sqlite`;
6. seal counts/digests and write a source receipt;
7. close the MySQL transaction;
8. emit a run journal and plan-vs-actual resource report;
9. optionally compare the document digest with the existing eager connector on a frozen fixture.

Acceptance criteria:

- no all-facet map;
- no corpus-sized document slice;
- exact current document IDs/text/metadata/digest;
- cancellation and crash-safe relation transactions;
- deterministic resume from the sealed source artifact;
- hard-limit test at 256 MiB on production-shaped source export;
- plan report contains no unknown/eager state.

Once this exists, the current chunk/representation logic can be adapted one unit at a time, and the already-proven RagKit BuildStream remains the downstream sink during transition.

20. Acceptance criteria for the full foundation

Correctness

- One source snapshot supplies all MySQL connector reads.
- Canonical relation and bundle identities are deterministic and versioned.
- Every parent/foreign-key/order/uniqueness invariant is checked at admission or seal.
- Existing bundle identity is preserved until an explicit migration.
- Partial runs cannot be mistaken for sealed, published, or active generations.

Memory and resources

- Every production node has a non-unknown retained-state class.
- No corpus-scale payload slice exists in extraction, chunking, representation, embedding, verification, inspection, or serving.
- Cardinality-bounded maps have configured ceilings and measured byte models.
- Build and query reservations are enforced at runtime.
- Writable scratch and remote immutable storage are separated.
- Hard cgroup tests demonstrate headroom for build, verify, startup, and concurrent serving.

Extensibility

- New source, transform, index, validator, or publisher implementations register through descriptors and conformance suites.
- Product algorithms remain in CoinVault; generic execution remains in RagKit.
- YAML can configure registered components but cannot execute arbitrary code.
- A new optimization dimension is a plan patch, not a new giant command branch.

Operations

- One run event stream feeds local logs, EMF, artifacts, and dashboards.
- Resume occurs only at durable checkpoints.
- Publication is receipt-last and create-only.

- Activation is CAS-based and rollback preserves evidence.
- Full verification and startup verification are explicit profiles.

Serving

- Bundle content opens read-only and does not load the corpus into heap.
- Candidate authorization precedes fusion, external reranking, and hydration.
- Retrieval, reranker, hydration, and evidence limits are compiled and enforced.
- Aggregate query concurrency is admitted against a service-wide resource budget.
- Tool results carry plan, bundle, policy, and resource identities.

21. Risks and anti-patterns

21.1 Building a universal workflow engine

Do not add distributed scheduling, arbitrary DAG services, UI orchestration, or a database-backed control plane to solve a local batch execution problem. EventBridge/Batch or River owns durable job execution. The kernel owns deterministic in-job dataflow, resources, artifacts, and receipts.

21.2 A goroutine per node

This creates hidden queues and makes the memory proof depend on runtime timing. Default to synchronous pull and sequential heavy sinks.

21.3 One-row-at-a-time dogma

Record iteration can create call and transaction overhead. Use bounded vectorized batches and document/group-local windows. The requirement is bounded ownership, not batch size one.

21.4 GOMEMLIMIT as the design

A soft runtime limit is a guard and calibration tool. It does not account for all cgroup memory and cannot fix an algorithmic live set.

21.5 Measuring only terminal RSS

Transient peaks caused the actual failures. Use stage/event and cgroup peak evidence.

21.6 Writing staging SQLite on EFS

Active SQLite transactions and network filesystem semantics are an avoidable risk. Build locally, close, verify, then publish immutable files.

21.7 Keeping two semantic sources

Do not let JSON arrays, staging tables, content SQLite, and backend manifests each become independently authoritative. Define canonical relations and derived projections.

21.8 Hiding sorts and maps inside transforms

A component that materializes and sorts all input has changed the physical plan. It must declare an external/eager state contract and be rejected or explicitly planned.

21.9 Unlimited service concurrency

Bounded requests multiplied by unbounded concurrent requests are not a bounded service.

21.10 Hot reload before release custody

Atomic channel activation plus process restart is simpler and safer initially. Hot reload adds handle lifetimes, concurrent generations, and rollback races.

21.11 A Turing-complete YAML DSL

Keep algorithms in code. The DSL selects registered components, wiring, policies, and bounds.

21.12 Treating telemetry as authorization

An EMF event saying “verification complete” does not make an artifact publishable. Only a validated receipt and state transition does.

22. Final recommendation

CoinVault should stop treating memory-bounded indexing and serving as a sequence of local repairs. The repairs have identified the reusable mechanisms already:

- ordered cursor admission;
- bounded batches;
- SQLite spill/materialization;
- canonical streaming folds;
- sequential backend projections;
- compact identity verification;
- immutable content-addressed artifacts;
- read-only content hydration;
- bounded top-k retrieval;
- pre-fusion authorization;
- resource telemetry and cgroup evidence.

The next step is to make those mechanisms a coherent executable contract.

Build a small pipeline compiler and runtime around typed relations, ownership, order, retained-state bounds, resource leases, effects, checkpoints, and receipts. Use it first to stream the MySQL snapshot into a sealed document relation. Then move chunking and representations upstream of the existing RagKit stager, consolidate verification/inspection over common readers, correct scratch/publication placement, and add query admission.

The resulting system will be easier to reason about because every stage answers the same questions:

What typed data do you consume and produce?
In what order?
What do you retain, and what bounds it?
What may you spill and where?
What external effects do you perform?
What identity proves the output?
Where can the run resume?
What receipt makes the result safe to use?

That is the missing foundation. Once it exists, memory optimization becomes a compiler and cost-model concern, new RAG dimensions become composable plan plugins, and the CoinVault end-to-end path—from MySQL to `knowledge_search`—can evolve without another proliferation of unrelated commands and bespoke operational harnesses.

Appendix A. Suggested Go types

```
package pipeline

type BoundClass string

const (
    BoundConstant BoundClass = "constant"
    BoundDimension BoundClass = "dimension"
    BoundBatch     BoundClass = "batch"
    BoundWindow    BoundClass = "window"
    BoundTopK      BoundClass = "topk"
    BoundIdentity  BoundClass = "identity"
    BoundExternal  BoundClass = "external"
    BoundEager     BoundClass = "eager"
    BoundUnknown   BoundClass = "unknown"
)

type StateBound struct {
    Class      BoundClass
    Items      int64
    Bytes      int64
    Formula    string
    Cardinality string
}

type Reservation struct {
    HeapBytes      int64
    ExternalBytes  int64
    ScratchBytes   int64
    FileDescriptors int
    Goroutines     int
    ProviderCalls  int
    InputTokens    int
    OutputTokens   int
    UnknownMarginBytes int64
}

type OrderKey struct {
    Field      string
    Direction  string
}

type OrderContract struct {
    Requires []OrderKey
    Produces []OrderKey
    Stable   bool
}

type Effect string

const (
    EffectMySQLRead      Effect = "mysql_read"
    EffectProviderCall   Effect = "provider_call"
    EffectScratchWrite    Effect = "scratch_write"
    EffectPublish        Effect = "publish"
    EffectActivate       Effect = "activate"
```

```

)

type RestartContract struct {
    Replayable      bool
    CheckpointKind  string
    IdempotencyKey  string
}

type OperatorDescriptor struct {
    Kind            string
    Version         string
    Deterministic   bool
    InputSchema     string
    OutputSchema    string
    Order           OrderContract
    Expansion       string
    Retained        StateBound
    Reservation     Reservation
    Effects         []Effect
    Restart         RestartContract
}

```

Appendix B. Example compiled node

```

{
  "id": "embed-representations",
  "component": "ragkit.embed/v2",
  "inputs": ["representation@sha256:..."],
  "outputs": ["vector"],
  "order": {
    "requires": ["representation_id"],
    "produces": ["representation_id"],
    "stable": true
  },
  "state_bound": {
    "class": "batch",
    "formula": "2 * batch_items * (max_text_bytes + dimensions*4)",
    "bytes": 150994944
  },
  "effects": ["provider_call", "scratch_write"],
  "checkpoint": "vector-batch-commit",
  "semantic_identity": "sha256:...",
  "physical_identity": "sha256:..."
}

```

Appendix C. Current source map

CoinVault source and serving

- internal/knowledgebuild/connectors.go — eager product facet map, ordered MySQL source queries, document normalization, scopes/roles, live-fact exclusions.
- Historical internal/knowledgebuild/build.go and commit 1a3a5c5 — transition from eager vector accumulation to indexbundle.BuildStream, while still staging complete document/chunk/representation slices.

- `cmd/coinvault/cmds/knowledge.go` — build, verify, inspect, package/export, eval, sweep, judge, memory sinks, and related command orchestration.
- `internal/knowledge/service.go` — schema-v2 service open, bounded retrieval, authorization-before-fusion, reranking and hydration.
- `internal/knowledge/content_lookup.go` — bounded content-store lookups by candidate/final IDs.
- `internal/knowledge/tool.go` — bounded `knowledge_search`, evidence ledger, semantic runtime identities.
- `internal/knowledge/runtime_config.go` — shared production/evaluator tool composition.
- `internal/webchat/server/server.go` and `cmd/coinvault/cmds/serve.go` — application composition and tool registration.

RagKit mechanisms

- `rag/indexbundle/build_stream.go` — temporary generation, SQLite staging, seal, backend projections, atomic publication.
- `rag/indexbundle/staging_kernel.go` — phase-ordered validated batch admission and ordered producers.
- `digest/digest.go` — canonical streamed JSON-sequence identities.
- `rag/indexbundle/verify_stream.go` — bounded payload verification and compact identity state.
- `rag/indexbundle/open.go` — full verify followed by read-only content/lexical/vector handles.
- `rag/content/sqlite` — bounded document/chunk/metadata lookups.
- `rag/lexical/bleve/index.go` — streamed record build and paged content inspection.
- `rag/vector/sqliteexact/index.go` — streamed build/inspection and bounded top-k exact search.

Operational evidence

- `COINVAULT-INDEX-00M-001/reference/01-investigation-diary.md` — failure reproduction, staged builder, real build, verifier measurements, streaming verifier proof.
- `GEC-RAG-REFRESH-001/design-doc/01-...md` — source snapshot, refresh state, publication, activation, rollback, and AWS orchestration design.
- `COINVAULT-INDEX-METRICS-001/design-doc/01-...md` — sampler, resource event, EMF, dashboard, and alarm design.
- `COINVAULT-PROD-002/design-doc/01-...md` — current production topology, schema-v2 serving, immutable release tuple, and operations.

Appendix D. Design influences and references

1. Alok Aggarwal and Jeffrey S. Vitter, “The Input/Output Complexity of Sorting and Related Problems,” *Communications of the ACM* 31(9), 1988, DOI 10.1145/48529.48535. Establishes the external-memory/I/O model and optimal block-oriented sorting bounds.
2. Hong Jia-Wei and H. T. Kung, “I/O Complexity: The Red-Blue Pebble Game,” STOC 1981, DOI 10.1145/800076.802486. Models live fast-memory versus external-memory movement on computation DAGs.
3. Goetz Graefe, “Volcano—An Extensible and Parallel Query Evaluation System,” *IEEE TKDE* 6(1), 1994, DOI 10.1109/69.273032. Influences the pull iterator, extensible operator, and mechanism/policy split.
4. Edward A. Lee and David G. Messerschmitt, “Synchronous Data Flow,” *Proceedings of the IEEE* 75(9), 1987. Influences declared rates, finite buffers, and static schedule analysis.
5. MySQL 8.4 Reference Manual, “Consistent Nonlocking Reads” and transaction isolation. Supports one read-only REPEATABLE READ snapshot for a generation.
6. SQLite documentation, “Atomic Commit in SQLite” and “Write-Ahead Logging.” Supports transactional materialization and warns about network-filesystem locking/WAL constraints.

7. Go runtime/debug.SetMemoryLimit documentation and the Go GC guide. Defines the soft Go-runtime memory limit and its exclusions; supports using GOMEMLIMIT as a guard rather than an algorithmic bound.
8. RagKit and CoinVault source and investigation artifacts listed in Appendix C.

Appendix E. Concrete current-to-target refactoring map

Current code or behavior	Target form	Primary reason
LoadProductDocuments(... *sqlx.DB) []rag.Document loadProductFacets() map[int64][]productFacet LoadCategoryDocuments() []rag.Document SQLDocDocuments() []rag.Document	snapshot-owned ordered product cursor sorted facet cursor plus merge-group join ordered category cursor small deterministic cursor	one source snapshot; no corpus payload slice retain one bounded product group common source contract and bounded payload common relation semantics
concatenate and sort.Slice all documents json.MarshalIndent over the complete corpus StripFurniture([]Document)	keyed relation writer or k-way source merge streaming canonical encoder/export projection document-local transform with audit relation	explicit external ordering remove duplicate full encoded buffer avoid a second corpus slice
chunking.Apply over all documents raw and breadcrumb representation slices followed by Compose stageBatches over complete docu- ment/chunk/representation slices item-count-only batch limits	per-document chunk emitter fixed-fanout per-chunk operator direct envelope transactions into relation store item, byte, token, and expected-output limits	bound liveness by one document static rate and immediate release move the external-memory boundary to the source handle large records and vector responses
writable staging under the bundle destination separate build/verify/open/inspect traversals full release verification on every service start imperative serving fields and local limits exact vector backend without declared work budget package-specific observer callbacks and samplers giant knowledge command implementation	explicit local scratch workspace common sealed relation and verified bundle cursor APIs explicit full/receipt/audit verification profiles immutable compiled retrieval plan plus request governor backend capability/cost contract common run event stream with sink adapters thin bindings over plan/build/verify/publish/activate operations	avoid network-filesystem writer behavior one set of order, identity, and resource contracts preserve safety without mandatory full startup scan bound aggregate work under concurrency distinguish heap bound from $O(ND)$ query work remove parallel telemetry orchestration reduce lifecycle duplication

A minor correctness defect in the supplied serving code should be repaired during this work: Service.HasDocument currently reports success whenever Content.Documents returns no error, without checking that the requested document was actually returned. Typed lookup contracts should distin-

guish a successful empty lookup from a found value.

Appendix F. Retention and work ledger

Component	Current or historical state	Boundedness class	Required target
product facet loader	all facet strings grouped by product	eager/corpus payload	window with maximum group bytes
source connectors	all admitted documents	eager/corpus payload	item/window plus external relation
corpus writer	full encoded JSON buffer	eager/corpus payload	one record plus encoder buffer
furniture stripping	possible second document corpus	eager/corpus payload	document-local
chunking	all chunks	eager/corpus payload	document-local bounded fanout
representation composition	all raw and breadcrumb records	eager/corpus payload	fixed fanout per chunk
embedding request	one provider batch	batch	weighted item/byte/token/output batch
historical vector collection	every vector coordinate	eager Theta (ND)	one batch then external relation
construction relation	all semantic records on disk	external	scratch quota and ordered cursors
streaming verifier chunks	one payload plus chunk ID/digest maps	identity 0 (N)	declared cardinality ceiling or external merge
streaming verifier representations	one payload plus representation ID map	identity 0 (N)	declared cardinality ceiling or external merge
Bleve verification	one deterministic page	batch	retain
vector verification	one row/vector	dimension	retain
lexical results	configured depth	top-k	request budget
exact vector search	query vector plus top-k heap; work 0 (ND)	dimension/top-k plus unbounded work with N	query work budget and admission
fusion/interleave	variants times depth candidate IDs	batch/top-k	request budget
reranker	configured candidate text	weighted batch	global provider/native sub-pool
final hydration	final documents/chunks only	batch	retain with request memo
evidence ledger	items and runes	byte budget	retain
aggregate serving	per-request state times concurrency	implicit	global weighted admission

Appendix G. Plan and code-review checklist

- ☐ All MySQL reads for one generation share one read-only consistent snapshot.
- ☐ Every source has a stable complete order and maximum record size.

- ☐ Every join/group has a bounded side, bounded group, or external strategy.
- ☐ Every fanout is exact/bounded or externally materialized.
- ☐ No production operator is classified eager or unknown.
- ☐ Batches are bounded by bytes and provider constraints, not only item count.
- ☐ Every retained buffer has a lease, fixed baseline, or explicit external materialization.
- ☐ Native, mmap, and page-cache margins are declared and measured.
- ☐ Scratch high-water fits the deployment volume with headroom.
- ☐ Order and canonical digest grammar are explicit.
- ☐ Side effects are idempotent/content-addressed or occur after a durable checkpoint.
- ☐ Partial artifacts cannot be discovered as sealed, published, or active generations.
- ☐ Publication and activation are separate, audited, and reversible.
- ☐ Full release verification and startup acceptance policies are explicit.
- ☐ Aggregate service memory and work are bounded at maximum admitted concurrency.
- ☐ Authorization precedes external text disclosure and follows final ranking as defense in depth.
- ☐ Differential, metamorphic, corruption, crash, and hard-limit tests pass.
- ☐ Source, component, logical plan, physical plan, relation, artifact, and runtime identities are recorded.
- ☐ The exact implementation and proof artifacts are reproducible from one merged commit or release tag.

Appendix H. Additional references

9. John D. C. Little, "A Proof for the Queuing Formula: $L = \lambda W$," *Operations Research* 9(3), 383–387, 1961, DOI 10.1287/opre.9.3.383.
10. Reactive Streams, official specification for asynchronous stream processing with non-blocking backpressure. Its demand protocol motivates bounded asynchronous edges, while CoinVault's default executor should remain synchronous pull except at explicit islands.
11. MySQL Reference Manual, `START TRANSACTION ... READ ONLY, WITH CONSISTENT SNAPSHOT`, and InnoDB consistent nonlocking reads. These support one internally consistent extraction snapshot; freshness and replica lag remain separate promotion checks.
12. SQLite Project, "Atomic Commit in SQLite." SQLite transactions suit local construction materialization; multi-file publication still needs an immutable higher-level commit receipt.